

APL (Amharic Programming Language)

1. Overview

Programming languages are usually designed using English keywords such as if, while, and return. For beginners whose first language is not English, learning programming often requires learning both programming concepts and unfamiliar English vocabulary at the same time.

To address this challenge, We designed **APL (Amharic Programming Language)**, a small educational programming language that uses Amharic keywords while translating programs into Python. The objective is not to replace existing programming languages but to provide a learning tool that allows beginners to understand programming concepts through familiar language constructs.

APL follows a source-to-source translation approach. Programs written in APL are first analyzed by a lexer and parser, represented internally as an Abstract Syntax Tree (AST), and then translated into executable Python code.

The language intentionally remains small and focused so that new programmers can understand its syntax quickly.

2. Lexical Structure

APL supports Unicode characters and therefore allows both Amharic and English identifiers.

Examples of valid identifiers:

ቁጥር ውጤት = 10

ቁጥር total = 20

Whitespace is ignored except when separating tokens. Strings are enclosed using double quotation marks.

Example: የፍ("ሰላም ")

Numbers currently support integer values.

3. Keywords

The language defines a small set of reserved words.

Keyword	Purpose
ቁጥር	Variable declaration
የፍ	Display output

ከሆነ	Conditional statement
ካልሆነ	Alternative branch
ሲሆን	Repetition loop
ተግባር	Function definition
መልስ	Return value
ጀምር	Beginning of a block
ጨርስ	End of a block

These keywords were chosen because their meaning closely reflects their programming purpose.

4. Variables and Expressions

Variables are introduced using the keyword ቁጥር.

Example:

ቁጥር $x = 10$

ቁጥር $y = 5$

Variables may later be updated: $x = x + 1$

APL supports the following arithmetic operations: + , - , * , /

Example: ቁጥር total = $x + y$

Expressions may appear in assignments, conditions, function calls, and output statements.

5. Conditional Statements

APL provides decision-making through the ከሆነ and ካልሆነ constructs.

Example:

ከሆነ $x > 10$:

ጀምር

ፃፍ("ትልቅ")

ጨርስ

ካልሆነ:

```
ጀምር
ፃፍ("ትንሽ")
ጨርስ
```

Comparison operators include: `==` , `<` , `>`

The translator converts these statements directly into Python conditional blocks.

6. Iteration

APL currently supports iteration through a while-style loop using the keyword ሲሆን.

Example:

```
ቁጥር x = 1
```

```
ሲሆን x < 5:
```

```
ጀምር
```

```
ፃፍ(x)
```

```
x = x + 1
```

```
ጨርስ
```

The loop executes repeatedly while the condition remains true.

7. Functions

Reusable computations are represented through functions.

Example:

```
ተግባር ድምር(a,b):
```

```
ጀምር
```

```
መልስ a + b
```

```
ጨርስ
```

Function invocation:

```
ፃፍ(ድምር(3,4))
```

Parameters are passed inside parentheses and return values are produced using the መልስ keyword.

8. Program Blocks

A notable design choice in APL is the use of explicit block delimiters:

```
ጀምር
```

...

ጨርስ

Many modern languages rely on indentation or braces. For beginner programmers, indentation mistakes can be confusing. Explicit block markers make program structure visible and reduce common syntax errors.

Example:

ከሆነ $x > 5$:

ጀምር

ፃፍ("OK")

ጨርስ

This approach also simplifies parsing because block boundaries are explicitly identified.

9. Translation Process

The compiler is divided into three stages:

1. **Lexical Analysis** – converts source code into tokens.
2. **Parsing** – validates syntax and constructs an AST.
3. **Code Generation** – produces equivalent Python code.

For example:

APL:

ፃፍ("ሰላም")

Generated Python:

```
print("ሰላም")
```

This architecture satisfies the requirement of performing structured translation rather than simple text replacement.

10. Conclusion

APL demonstrates how programming concepts can be introduced through a local language. By combining Amharic keywords, explicit block structures, Unicode support, and Python code generation, the language provides an accessible environment for beginner programmers while remaining technically structured through lexical analysis, parsing, and AST-based translation.